FlyWithLua Script Subfolders & SkunkCrafts Updater Guide

FlyWithLua 2.8.20+ -- SkunkCrafts Updater v4.2.7+

Official Architecture Reference, User Guide, and Developer Best Practices

> ***Date:*** *October 2026*

> ***Target Release:*** *FlyWithLua 2.8.20+ (Built against Dear ImGui 1.92.8 & ImgWindow v2.2.2)*

> ***Audience:*** *End-Users, Aircraft Developers, and Lua Script Authors*

---

1. Executive Summary & Background

Historically, FlyWithLua loaded script files strictly from the root of `Resources/plugins/FlyWithLua/Scripts/`. Any `.lua` files placed inside subdirectories were completely ignored during simulator startup.

While this flat layout originally prevented companion or private helper files from executing unintentionally, it created major challenges for modern addon ecosystems:

1. **Root Script Clutter:** Complex addons with multiple auxiliary `.lua` files, textures, sound effects, and configuration files flooded the top-level `Scripts/` directory.
2. **SkunkCrafts Updater Collisions:** The **SkunkCrafts Updater** requires its tracking manifest, `skunkcrafts_updater.cfg`, to live in the same directory as the payload files it manages. In a flat `Scripts/` directory, only one single addon could ever use SkunkCrafts before file name collisions broke updating for all other addons.

Starting with **SkunkCrafts Updater v4.2.7** and **FlyWithLua 2.8.20+**, FlyWithLua now natively supports **controlled 1-level script subfolders**. Furthermore, SkunkCrafts Updater v4.2.7 no longer recognizes `.cfg` files placed at the top level of `Scripts/` -- preventing any single addon from monopolizing the root directory.

Developers can now package their entire addon into a clean, self-contained subfolder that SkunkCrafts tracks, verifies, and updates in-place.

---

2. End-User Installation Guide

If you are an X-Plane pilot installing an addon or script designed for FlyWithLua and SkunkCrafts Updater, follow these simple rules:

How to Install

1. **Keep the Folder Intact:** When you download or extract a subfolder-compatible package, do **not** pull individual `.lua` files out of their parent folder.
2. **Place Directly Inside `Scripts/`:** Move or copy the entire folder directly into:

```
X-Plane 12/Resources/plugins/FlyWithLua/Scripts/<Addon_Folder_Name>/
```

3. **Flexible Folder Naming (No Redundant Nesting):** For SkunkCrafts-managed packages, the outer folder name does **not** need to match the script name. For instance, rather than creating awkward redundant paths like `Simple_Ground_Equipment_and_Services/Simple_Ground_Equipment_and_Services/`, you can use a clean, concise folder name like `SGES/`.
4. **Verify Contents:** Look inside the folder. It should contain either:

* A script whose name matches the folder (e.g., `Scripts/MyScript/MyScript.lua`), OR
* A `skunkcrafts_updater.cfg` file alongside the addon's main `.lua` scripts.

Directory Structure Example

```
X-Plane 12/
■■■ Resources/
    ■■■ plugins/
        ■■■ FlyWithLua/
            ■■■ Scripts/
                ■■■ classic_script.lua                    <-- Runs (Root script)
                ■■■ another_root_mod.lua                  <-- Runs (Root script)
                ■
                ■■■ SGES/                                 <-- Clean SkunkCrafts Subfolder
                ■   ■■■ skunkcrafts_updater.cfg            <-- SkunkCrafts marker
                ■   ■■■ Simple_Ground_Equipment_and_Services.lua <-- Runs (Entry point)
                ■   ■■■ Simple_Ground_Equipment_and_Services/ <-- Nested companion files (Protected)
                ■       ■■■ Front_Line.lua
                ■       ■■■ Threat.lua
                ■       ■■■ Sounds/
                ■
                ■■■ MyCustomHUD/                          <-- Standalone Subfolder Mod
                    ■■■ MyCustomHUD.lua                   <-- Runs (Base name matches folder)
                    ■■■ lib_helper.lua                    <-- Private helper (NOT auto-loaded)
```

3. Discovery Architecture & Loading Rules

FlyWithLua inspects `Scripts/` and immediate 1-level subdirectories during startup or script reload. Discovery operates under three definitive rules:

Rule 0: Classic Root Scripts (100% Backward Compatible)

Any valid `.lua` file placed directly in `Scripts/` runs exactly as it always has. Existing scripts require zero modifications.

Rule 1: Standalone Named Subfolder (Single-Script Mod)

If a subfolder does **not** contain `skunkcrafts_updater.cfg`, FlyWithLua only loads the script whose base name matches the folder name:

* `Scripts/WeatherRadar/WeatherRadar.lua` -> **LOADED**
* `Scripts/WeatherRadar/radar_math.lua` -> **SKIPPED** (Can be loaded via `dofile()` or `require()`)
* `Scripts/MyMod.lua/MyMod.lua` -> **LOADED** (Folder extensions like `.lua` are stripped before matching)

This allows standalone script developers to package private modules, images, and config files cleanly in a dedicated directory without auxiliary files executing prematurely.

Rule 2: SkunkCrafts Updater Subfolder (Multi-Script Addon)

If an immediate subfolder contains `skunkcrafts_updater.cfg` (case-insensitive):

* **All valid `.lua` files located directly inside that 1-level subfolder are loaded as top-level entry points.**
* Non-script files (`.cfg`, `.txt`, `.json`, `.ini`) are ignored by the loader.
* **Sub-subfolders Are Strictly Protected:** Files located in nested subdirectories (e.g., `Scripts/SGES/Simple_Ground_Equipment_and_Services/Front_Line.lua` or `Scripts/MyAddon/modules/extra.lua`) are **never** auto-loaded, ensuring companion and private libraries remain isolated.

4. Defensive Safeguards & Troubleshooting

To prevent accidents, broken updates, and conflicts, FlyWithLua implements several defensive layers:

1. Global INI Master Toggle

In `Resources/plugins/FlyWithLua/fwl_prefs.ini`:

```
;EnableScriptSubfolders = 0 Disabled (strictly load Scripts/ root files only)
;EnableScriptSubfolders = 1 Enabled (allow 1-level subfolders for standalone scripts and SkunkCrafts Updater)
EnableScriptSubfolders = 1
```

Setting `EnableScriptSubfolders = 0` acts as an immediate master killswitch, strictly reverting FlyWithLua to classic root-only loading without code changes.

2. Folder Exclusion Markers

FlyWithLua will immediately skip any subfolder that contains any of the following marker files:

* `.no_run`
* `.norun`
* `.disabled`
* `no_run.txt`

Additionally, any directory whose name ends with `(disabled)` or `[disabled]` is skipped automatically.

3. Root Script Shadow Guard

If a script already exists at the root level (e.g. `Scripts/steve-cool.lua`), FlyWithLua will **not** auto-load `Scripts/steve-cool/steve-cool.lua` under Rule 1. This prevents companion data/asset subfolders that share the root script's name from accidentally double-executing.

4. Quarantine Recovery & Troubleshooting

If a script in a subfolder encounters a syntax error or crashes during startup:

* FlyWithLua replicates the subfolder hierarchy inside `Scripts (Quarantine)/` before moving the offending script.
* **First-Time Installation Check:** If an addon script is quarantined upon first install, always inspect `Log.txt` or `FlyWithLua_Debug.txt`. Because the script now resides 1 level deeper in the filesystem, legacy scripts that hardcode relative paths (e.g. `../../../../`) may fail to find external scenery or assets.
* Point the script to the proper globals (`CURRENT_SCRIPT_DIRECTORY` for its own folder, or `ROOT_SCRIPT_DIRECTORY` for root `Scripts/`), or report the issue to the author for a permanent update.
* When fixed or restored, FlyWithLua returns the script directly to its original subfolder.

---

5. Lua Directory Path Globals

FlyWithLua provides comprehensive globals to give script authors clean, reliable access to files:

Global Variable	Type	Behavior & Purpose
`SCRIPT_DIRECTORY`	String	**Dynamic by Default (Option A):** Points to the currently executing script's subfolder (with trailing slash) while loading. Automatically captured as a file-local upvalue so callbacks permanently retain their path. Restored to root `Scripts/` when startup finishes.
`ROOT_SCRIPT_DIRECTORY`	String	**Constant Static Root:** Always points strictly to `Resources/plugins/FlyWithLua/Scripts/`. Use this whenever your script needs to verify or interact with other installed scripts or root resources.
`CURRENT_SCRIPT_DIRECTORY`	String	**Explicit Script Folder:** Points to the directory containing the currently running script (e.g. `.../Scripts/SGES/` for subfolders, or `.../Scripts/` for root scripts).
`SCRIPT_SUBDIRECTORY`	String	**Relative Subfolder Path:** Contains the relative folder name with trailing slash (e.g. `"SGES/"` for subfolder scripts, or `""` for root scripts).

Global Variable	Type	Behavior & Purpose
`SYSTEM_DIRECTORY`	String	**X-Plane Root:** Points to the main X-Plane installation directory (e.g. `.../X-Plane 12/`).
`PLUGIN_MAIN_DIRECTORY`	String	**FlyWithLua Root:** Points to `.../Resources/plugins/FlyWithLua/`.

---

6. Developer Best Practices: Dos and Don'ts

DO: Use `SYSTEM_DIRECTORY` for X-Plane Root Paths

Do this:

```
-- Clean, reliable across root and subfolders
local scenery_ini = io.open(SYSTEM_DIRECTORY .. "Custom Scenery/scenery_packs.ini", "r")
```

Don't do this:

```
-- FRAGILE! Hardcodes 4 directory levels up from Scripts/
-- Breaks immediately if your script is moved into a subfolder!
local scenery_ini = io.open(SCRIPT_DIRECTORY .. "../../../../Custom Scenery/scenery_packs.ini", "r")
```

DO: Place Private Modules in a Nested Folder Under SkunkCrafts

When your addon includes `skunkcrafts_updater.cfg`, Rule 2 causes FlyWithLua to run **all** `.lua` files in that immediate subfolder.

* **Good Layout:**

```
  Scripts/MyAircraftMod/
  ■■■ skunkcrafts_updater.cfg
  ■■■ MyAircraftMod.lua          <-- Auto-runs (Entry point)
  ■■■ modules/                   <-- Nested folder! (Safe from auto-run)
      ■■■ fuel_system.lua        <-- Loaded via require() or dofile()
      ■■■ navigation_math.lua    <-- Loaded via require() or dofile()
```

* **Dangerous Layout:**

```
  Scripts/MyAircraftMod/
  ■■■ skunkcrafts_updater.cfg
  ■■■ MyAircraftMod.lua          <-- Auto-runs
  ■■■ fuel_system.lua            <-- Auto-runs independently at startup! (May crash if it needs setup)
  ■■■ navigation_math.lua        <-- Auto-runs independently at startup!
```

DO: Use `ROOT_SCRIPT_DIRECTORY` to Check for Other Plugins

If your script wants to detect whether an optional utility (like an FPS counter or another script) is installed:

```
local other_script = io.open(ROOT_SCRIPT_DIRECTORY .. "fps_counter.lua", "r")
if other_script then
    io.close(other_script)
    -- Utility is present!
end
```

DO: Use Clean Relative Paths Inside Your Addon

With Option A active by default, you can simply write:

```
dofile(SCRIPT_DIRECTORY .. "modules/fuel_system.lua")
local bg_image = float_wnd_load_image(SCRIPT_DIRECTORY .. "textures/hud.png")
```

This code works identically whether your addon is installed inside a subfolder or directly in classic root `Scripts/`!

---

7. Preference Configuration: Option A vs Option B

In `Resources/plugins/FlyWithLua/fwl_prefs.ini`, authors and advanced users can configure how `SCRIPT_DIRECTORY` behaves:

```
;SubfolderSetsScriptDirectory = 0 Option B: SCRIPT_DIRECTORY strictly points to Scripts/ root (use CURRENT_SCRIPT_DIRECTOR
;SubfolderSetsScriptDirectory = 1 Option A (default): SCRIPT_DIRECTORY dynamically points to the script's subfolder during
SubfolderSetsScriptDirectory = 1
```

* **Option A (`1`, Default):** `SCRIPT_DIRECTORY` dynamically points to the script's subfolder during loading and is captured by subfolder scripts. Existing legacy packages run out-of-the-box with zero code changes.
* **Option B (`0`, Legacy Fallback):** `SCRIPT_DIRECTORY` is kept static at root `Scripts/`. Authors must explicitly use `CURRENT_SCRIPT_DIRECTORY` to find their companion files.